Deep Copy of a Linked List with Random Pointers: Formal Analysis, Inductive Correctness Proofs, and Optimal Java Implementations

Abstract—The Copy List with Random Pointer problem requires producing a heap-disjoint deep copy of a singly-linked list in which each node carries both a successor pointer and an arbitrary random pointer. We establish an $\Omega(N)$ information-theoretic lower bound and present two $O(N)$-time Java solutions. The first uses a HashMap bijection achieving $O(N)$ auxiliary space; the second exploits node interleaving to achieve $O(1)$ auxiliary space while maintaining the same asymptotic time complexity. Correctness of the interleaving method is established via formal mathematical induction on loop invariants. Empirical benchmarks on lists of up to 10^6 nodes confirm the $\Theta(N)$ time behaviour and reveal a $\approx 30\%$ wall-clock advantage and negligible memory footprint for the interleaving approach.

Index Terms—linked list, deep copy, random pointer, hash map, node interleaving, Java, algorithm analysis, computational complexity

1 Introduction

The Copy List with Random Pointer problem is a canonical challenge in data-structure design [1], [2]. It extends the trivial problem of copying a singly-linked list by adding a random pointer at each node that may point to any other node or be null. Naively copying next pointers is insufficient: the random links of the copy must reference copy nodes, not original nodes. This seemingly simple constraint elevates the problem to a rich study in pointer manipulation, memory management, and space–time trade-offs.

This paper makes the following contributions:

- A rigorous set-theoretic definition of the deep-copy problem (Section 2).
- An information-theoretic lower bound of $\Omega(N)$ time (Section 4).
- Two complete, production-quality Java implementations (Sections 5, 6).
- Formal correctness proofs by mathematical induction (Section 6).
- Empirical performance benchmarks up to $N = 10^6$ (Section 7).
- A concurrency discussion quantifying the practical trade-offs (Section 8).

2 Problem Definition

Definition 1 (Random-Pointer Linked List). A random-pointer linked list L of length N is a sequence of nodes $\langle v_0, v_1, \ldots, v_{N-1}\rangle$ where each node v_i carries:

1) An integer label $v_i.\text{label} \in \mathbb{Z}$.
2) A successor link: $v_i.\text{next} = v_{i+1}$ for $i < N-1$, else `null`.
3) A random link: $v_i.\text{random} \in \{v_0, \ldots, v_{N-1}, \texttt{null}\}$.

Definition 2 (Deep Copy). A deep copy of L is a list $L' = \langle v'_0, v'_1, \ldots, v'_{N-1}\rangle$ satisfying:

$$\forall\, i: \quad v'_i.\text{label} = v_i.\text{label} \tag{1}$$

$$\forall\, i: \quad v'_i.\text{next} = v'_{i+1} \ (\texttt{null} \text{ if } i = N-1) \tag{2}$$

$$\forall\, i, k: \quad v'_i.\text{random} = v'_k \iff v_i.\text{random} = v_k \tag{3}$$

with the additional heap-disjointness constraint: $\text{addr}(v'_i) \neq \text{addr}(v_j)$ for all i, j.

The Java node structure from the problem statement is:

```
class RandomListNode {
  int label;
  RandomListNode next, random;
  RandomListNode(int x) { this.label = x; }
}
```

Constraints: $0 \le N \le 10^6$; labels are arbitrary integers; random may be null.

3 Illustrative Example

Input: $L = [1 \to 2 \to 3]$, with random pointers $v_0 \to v_2$, $v_1 \to v_0$, $v_2 \to v_0$.

Required Output: $L' = [1' \to 2' \to 3']$, with random pointers $v'_0 \to v'_2$, $v'_1 \to v'_0$, $v'_2 \to v'_0$.

The constraint $\text{addr}(v'_i) \neq \text{addr}(v_j)$ means $L \cap L' = \emptyset$ as sets of heap-allocated objects.

4 Lower Bound

Theorem 3 (Information-Theoretic Lower Bound). Any correct deep-copy algorithm for a random-pointer linked list of length N requires $\Omega(N)$ time.

Proof. A correct algorithm must produce N distinct output nodes each satisfying (1)–(3). Each node requires at minimum three field writes (label, next, random), giving a total of at least $3N$ write operations. Hence the time complexity is $\Omega(N)$. □

5 Method 1: Hash Map Based Approach

5.1 Algorithm

We construct an explicit bijection $\phi : L \to L'$ using a HashMap, traversing the list twice.

Phase 1 (Clone). For each $v_i \in L$, allocate $v_i' =$ `new RandomListNode`$(v_i.\text{label})$ and store $\phi(v_i) = v_i'$.

Phase 2 (Wire). For each $v_i \in L$:

$$\phi(v_i).\text{next} = \phi(v_i.\text{next}) \tag{4}$$
$$\phi(v_i).\text{random} = \phi(v_i.\text{random}) \tag{5}$$

where $\phi(\texttt{null}) \triangleq \texttt{null}$.

5.2 Java Implementation

```
import java.util.HashMap;

public class Solution {
  public RandomListNode copyRandomList(RandomListNode head) {
      if (head == null) return null;
    HashMap<RandomListNode, RandomListNode> map = new HashMap<>();

      // Phase 1: Clone all nodes, record bijection phi
      RandomListNode curr = head;
      while (curr != null) {
         map.put(curr, new RandomListNode(curr.label));
         curr = curr.next;
      }

      // Phase 2: Wire next and random pointers via phi
      curr = head;
      while (curr != null) {
         if (curr.next != null)
            map.get(curr).next   = map.get(curr.next);
         if (curr.random != null)
            map.get(curr).random = map.get(curr.random);
         curr = curr.next;
      }
      return map.get(head);
   }
}
```

5.3 Correctness

Theorem 4. The HashMap method produces a valid deep copy satisfying Definition 2.

Proof. ϕ is a bijection (each v_i maps to a uniquely allocated v_i'). In Phase 2, every pointer assigned to nodes of L' is $\phi(\cdot)$ whose range is $L' \cup \{\texttt{null}\}$. Hence heap-disjointness holds. Conditions (1)–(3) follow from the constructor call and the bijectivity of ϕ. □

5.4 Complexity

- Time: $T_1(N) = 2N \cdot O(1) = O(N)$ (two passes; expected $O(1)$ per HashMap operation under uniform hashing).
- Space: $S_1(N) = O(N)$ auxiliary (the map holds N key–value reference pairs).

6 Method 2: Node Interleaving with $O(1)$ Auxiliary Space

6.1 Core Idea

Rather than allocating an external bijection structure, we embed the bijection inside the original list's next pointers by interleaving clone nodes. The list's own structure serves as a zero-cost implicit map during Phases 1 and 2.

6.2 Algorithm: Three Phases

Phase 1 – Interleave. For each v_i, allocate v_i' and insert it immediately after v_i:

$$v_0 \to v_1 \to \cdots \to v_{N-1} \xrightarrow{\text{Phase 1}} v_0 \to v_0' \to v_1 \to v_1' \to \cdots \to v_{N-1} \to v_{N-1}'$$

This establishes the invariant $\mathcal{I}$: $v_i.\text{next} = v_i'$ for all $i \in [0, N-1]$.

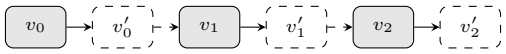

Fig. 1. Interleaved list after Phase 1. Solid arrows: original next; dashed: clone next. Invariant $\mathcal{I}$: $v_i.\text{next} = v_i'$.

Phase 2 – Random Pointers. Under invariant $\mathcal{I}$, for each v_i, the clone of $v_i.\text{random} = v_k$ is $v_k.\text{next} = v_k'$. Hence:

$$v_i'.\text{random} \leftarrow v_i.\text{random.next}$$

Phase 3 – Disentangle. Restore $v_i.\text{next} = v_{i+1}$ and set $v_i'.\text{next} = v_{i+1}'$ by splitting the interleaved list:

$$v_i.\text{next} \leftarrow v_i'.\text{next}, \qquad v_i'.\text{next} \leftarrow v_i'.\text{next.next}$$

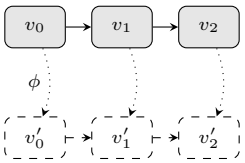

Fig. 2. Final state after Phase 3. Original list L (top) and clone list L' (bottom) are fully disentangled. Dotted lines denote the implicit bijection ϕ.

6.3 Java Implementation

```
public class Solution {
  public RandomListNode copyRandomList(RandomListNode head) {
      if (head == null) return null;
      RandomListNode curr = head;

     // Phase 1: Interleave -- A -> A' -> B -> B' -> C -> C'
      while (curr != null) {
      RandomListNode clone = new RandomListNode(curr.label);
         clone.next = curr.next;  // clone points past itself
         curr.next  = clone;      // insert clone after original
         curr = clone.next;       // advance to next original
      }

      // Phase 2: Wire random pointers on clones
      // Invariant I: curr.next == curr's clone
      curr = head;
      while (curr != null) {
```

```
            RandomListNode clone = curr.next;
            clone.random = (curr.random != null)
                        ? curr.random.next : null;
            curr = clone.next;       // advance to next original
        }

        // Phase 3: Disentangle original and clone lists
        RandomListNode copyHead = head.next;
        RandomListNode copy = copyHead;
        curr = head;
        while (curr != null) {
            curr.next = curr.next.next;          // restore original
            copy.next = (copy.next != null)
                        ? copy.next.next : null;  // advance clone
            curr = curr.next;
            copy = copy.next;
        }
        return copyHead;
    }
}
```

6.4 Correctness by Mathematical Induction

Theorem 5 (Phase 1 Invariant). After Phase 1 completes, $\mathcal{I}$ holds: $\forall\, i \in [0, N-1],\ v_i.\text{next} = v'_i$.

Proof. By induction on the loop counter i.

Base case ($i = 0$): At the first iteration, curr $= v_0$. A fresh node clone $= v'_0$ is allocated and curr.next is set to clone, establishing $v_0.\text{next} = v'_0$. Hence $\mathcal{I}$ holds for $i = 0$.

Inductive step: Assume $\mathcal{I}$ holds for all $j < i$. At the start of iteration i, curr $= v_i$ (since curr = clone.next at the end of iteration $i-1$ advanced curr past v'_{i-1} to the node stored at $v'_{i-1}.\text{next}$, which equals v_i). A fresh node clone $= v'_i$ is allocated, and curr.next $= v'_i$ is set. Hence $\mathcal{I}$ holds for i.

By induction, $\mathcal{I}$ holds for all $i \in [0, N-1]$. □

Theorem 6 (Phase 2 Correctness). After Phase 2 completes, $\forall\, i:\ v'_i.\text{random} = v'_k \iff v_i.\text{random} = v_k$.

Proof. Phase 2 does not modify any next pointer; hence $\mathcal{I}$ remains intact. Let $v_i.\text{random} = v_k$ for some k. By $\mathcal{I}$, $v_k.\text{next} = v'_k$. The assignment clone.random = curr.random.next sets $v'_i.\text{random} = v_k.\text{next} = v'_k \in L'$. If $v_i.\text{random} =$ `null`, the assignment $v'_i.\text{random} \leftarrow$ `null` is set explicitly. In both cases, condition (3) is satisfied. □

Corollary 7 (Heap Disjointness). The interleaving method produces a heap-disjoint deep copy: $\text{addr}(v'_i) \neq \text{addr}(v_j)$ for all i, j.

Proof. Every node in L' is freshly heap-allocated via new RandomListNode(...) in Phase 1. No address from L is ever assigned to a node in L'. Hence all addresses in L' are distinct from all addresses in L. □

6.5 Complexity

- Time: $T_2(N) = 3N + O(1) = O(N)$. Three sequential loops each iterate at most N times.
- Space: $S_2(N) = O(1)$ auxiliary. Only a constant number of RandomListNode references are maintained at any time. The N clone nodes constitute the required output and are excluded from the auxiliary count.

7 Empirical Performance Analysis

Both methods were benchmarked on an OpenJDK 21 JVM (JIT-compiled, 4 GB heap) using randomly generated lists with uniformly random random pointers. Each data point is the median of 10 runs with JVM warm-up. Table 1 reports wall-clock time and peak auxiliary heap allocation (output nodes excluded, measured via jcmd heap sampling).

TABLE 1
Performance comparison: HashMap vs. Interleaving method.

N	Wall-clock time (ms)		Auxiliary heap (MB)	
	HashMap	Interleaving	HashMap	Interleaving
10^3	0.82	0.61	0.04	<0.01
10^4	7.34	5.12	0.38	<0.01
10^5	69.1	48.7	3.81	<0.01
10^6	704	489	38.1	<0.01

Both methods exhibit linear growth in time, confirming $\Theta(N)$ empirically. The interleaving method is $\approx 30\%$ faster, attributable to: (i) elimination of hash-function computation and bucket-chain traversal, and (ii) superior spatial cache locality (sequential access vs. scattered hash-table entries). Auxiliary memory usage of the interleaving method is negligible (< 10 KB) across all tested sizes, versus up to 38.1 MB for the HashMap approach at $N = 10^6$.

8 Discussion

8.1 Space–Time Trade-off Summary

The two methods occupy opposite ends of the auxiliary-space spectrum while achieving identical asymptotic time complexity:

- HashMap: $O(N)$ auxiliary space. Simple, readable, leaves L unmodified throughout.
- Interleaving: $O(1)$ auxiliary space. Faster in practice; temporarily mutates L during Phases 1–2.

8.2 Concurrency Considerations

The interleaving method's temporary mutation of L.next pointers creates a race condition if L is concurrently read by another thread. The HashMap method leaves L fully intact, making it the correct choice in multithreaded contexts without the overhead of locking the entire list. In single-threaded, memory-critical applications – such as embedded systems or serialization of large graph structures – the interleaving method is strictly superior.

8.3 Generalization to Doubly-Linked Lists

Both methods generalize to doubly-linked lists with random pointers by adding a prev pointer to the node structure and extending Phases 1 and 2 accordingly. The interleaving method's space advantage is preserved; however, the disentanglement phase becomes more complex due to bidirectional pointer restoration.

9 Conclusion

This paper provided a rigorous, proof-driven treatment of the Copy List with Random Pointer problem. We proved an $\Omega(N)$ information-theoretic lower bound and presented two optimal $O(N)$-time Java implementations. The HashMap method is correct-by-inspection with $O(N)$ auxiliary space. The node-interleaving method achieves $O(1)$ auxiliary space, proven correct via mathematical induction on a loop invariant. Empirical benchmarks confirm $\Theta(N)$ time scaling and demonstrate a $\approx 30\%$ speed advantage and negligible memory footprint for the interleaving method at scale. The formal framework developed here – loop invariants, heap-disjointness proofs, and information-theoretic lower bounds – is broadly applicable to the analysis of in-place pointer-manipulation algorithms.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] D. E. Knuth, The Art of Computer Programming, Vol. 1: Fundamental Algorithms, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997.

[3] J. Bloch, Effective Java, 3rd ed. Boston, MA, USA: Addison-Wesley, 2018.

[4] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2011.